

AMATH 516: Homework 4
Sid Meka

1. **Exercise 5.11:** Establish the following properties of f_t .

1. Define the point $\bar{x}_t \in \mathbb{R}^n$ whose i^{th} coordinate is given by

$$\begin{cases} 1 - \frac{i}{t+1}, & \text{if } i = 1, \dots, t \\ 0, & \text{if } i = t+1, \dots, n \end{cases}$$

Using first order conditions for optimality, show that $\bar{x}_t$ is a minimizer of f_t with optimal value $f_t^* = \frac{\beta}{8}(-1 + \frac{1}{t+1})$.

We need to check that $\nabla f_t(\bar{x}_{t_1}, \dots, \bar{x}_{t_t}) = 0$, where $\bar{x}_{t_i}$ is the i^{th} coordinate of $\bar{x}_t$. Let's explicitly write the variable part:

$$f_t(z) = \frac{\beta}{8} \left(z_1^2 + \sum_{i=1}^{t-1} (z_i - z_{i+1})^2 + z_t^2 - 2z_1 \right)$$

Let $z = \bar{x}_t$, where $z = (z_1, \dots, z_t)$.

Let's compute partial derivatives with respect to z_j .

i. For $j = 1$, the partial derivative is:

$$\frac{\partial f_t}{\partial z_1} = \frac{\beta}{8}(2z_1 + 2(z_1 - z_2) - 2)$$

$$\frac{\partial f_t}{\partial z_1} = \frac{\beta}{4}(z_1 - 1 + z_1 - z_2)$$

$$\frac{\partial f_t}{\partial z_1} = \frac{\beta}{4}(2z_1 - z_2 - 1)$$

ii. For $1 < j < t$, the partial derivative is:

$$\frac{\partial f_t}{\partial z_j} = \frac{\beta}{8}(2(z_j - z_{j-1}) + 2(z_j - z_{j+1}))$$

$$\frac{\partial f_t}{\partial z_j} = \frac{\beta}{4}(2z_j - z_{j-1} - z_{j+1})$$

iii. For $j = t$, the partial derivative is:

$$\frac{\partial f_t}{\partial z_t} = \frac{\beta}{8}(2(z_t - z_{t-1}) + 2z_t)$$

$$\frac{\partial f_t}{\partial z_t} = \frac{\beta}{8}(2z_t - 2z_{t-1} + 2z_t)$$

$$\frac{\partial f_t}{\partial z_t} = \frac{\beta}{8}(4z_t - 2z_{t-1})$$

$$\frac{\partial f_t}{\partial z_t} = \frac{\beta}{4}(2z_t - z_{t-1})$$

So,

$$\begin{aligned} 2a_t - a_{t-1} &= 2 \cdot \frac{1}{t+1} - \frac{2}{t+1} \\ &= \frac{2}{t+1} - \frac{2}{t+1} \\ &= 0 \end{aligned}$$

So, in all coordinates, $\nabla f_t(\bar{x}_t) = 0$, so $\bar{x}_t$ is a critical point.
Since f_t is convex, $\bar{x}_t$ must be a global minimizer.

$$f_t^* = f_t(\bar{x}_{t_1}, \dots, \bar{x}_{t_t})$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{1}{2} \left(z_1^2 + \sum_{i=1}^{t-1} (z_i - z_{i+1})^2 + z_t^2 \right) - z_1 \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{1}{2} \left(\left(1 - \frac{1}{t+1} \right)^2 + \sum_{i=1}^{t-1} \left(\left(1 - \frac{i}{t+1} \right) - \left(1 - \frac{i+1}{t+1} \right) \right)^2 + \left(1 - \frac{t}{t+1} \right)^2 \right) - \left(1 - \frac{1}{t+1} \right) \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{1}{2} \left(\left(\frac{t}{t+1} \right)^2 + \sum_{i=1}^{t-1} \left(\frac{1}{t+1} \right)^2 + \left(\frac{1}{t+1} \right)^2 \right) - \left(\frac{t}{t+1} \right) \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{1}{2} \left(\left(\frac{t}{t+1} \right)^2 + (t-1) \left(\frac{1}{t+1} \right)^2 + \left(\frac{1}{t+1} \right)^2 \right) - \left(\frac{t}{t+1} \right) \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{1}{2} \left(\frac{t^2}{(t+1)^2} + \frac{t-1}{(t+1)^2} + \frac{1}{(t+1)^2} \right) - \left(\frac{t}{t+1} \right) \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{1}{2} \left(\frac{t^2 + t}{(t+1)^2} \right) - \left(\frac{t}{t+1} \right) \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{t^2 + t}{2(t+1)^2} - \frac{t}{t+1} \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{t^2 + t}{2(t+1)^2} - \frac{2t(t+1)}{2(t+1)^2} \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{t^2 + t}{2(t+1)^2} - \frac{2t^2 + 2t}{2(t+1)^2} \right)$$

$$f_t(z) = \frac{\beta}{4} \left(\frac{-t^2 - t}{2(t+1)^2} \right)$$

$$f_t(z) = \frac{\beta}{4} \left(-\frac{t^2 + t}{2(t+1)^2} \right)$$

$$f_t(z) = -\frac{\beta}{4} \left(\frac{t(t+1)}{2(t+1)^2} \right)$$

$$f_t(z) = -\frac{\beta}{8} \left(\frac{t}{(t+1)} \right)$$

$$f_t(z) = -\frac{\beta}{8} \cdot \frac{t}{t+1}$$

$$f_t(z) = \frac{\beta}{8} \cdot -\frac{t}{t+1}$$

$$f_t(z) = \frac{\beta}{8} \cdot \left(-\frac{t+1}{t+1} + \frac{1}{t+1} \right)$$

$$f_t(z) = \frac{\beta}{8} \cdot \left(-1 + \frac{1}{t+1} \right)$$

$$f_t^* = f_t(z)$$

$$f_t^* = f_t(\bar{x}_t)$$

$$f_t^* = \frac{\beta}{8} \cdot \left(-1 + \frac{1}{t+1} \right)$$

2. Taking into account the standard inequalities,

$$\sum_{i=1}^t i = \frac{t(t+1)}{2} \quad \text{and} \quad \sum_{i=1}^t i^2 \leq \frac{(t+1)^3}{3}$$

show the estimate $\|\bar{x}_t\|^2 \leq \frac{1}{3}(t+1)$.

$$\bar{x}_{t_i} = 1 - \frac{i}{t+1}$$

$$\|\bar{x}_t\|^2 = \sum_{i=1}^t \left(1 - \frac{i}{t+1} \right)^2$$

$$\|\bar{x}_t\|^2 = \sum_{i=1}^t \left(1 - \frac{2i}{t+1} + \frac{i^2}{(t+1)^2} \right)$$

$$\|\bar{x}_t\|^2 = \sum_{i=1}^t 1 - \sum_{i=1}^t \frac{2i}{t+1} + \sum_{i=1}^t \frac{i^2}{(t+1)^2}$$

$$\|\bar{x}_t\|^2 = t - \frac{2}{t+1} \sum_{i=1}^t i + \frac{1}{(t+1)^2} \sum_{i=1}^t i^2$$

$$\|\bar{x}_t\|^2 = t - \frac{2}{t+1} \left(\frac{t(t+1)}{2} \right) + \frac{1}{(t+1)^2} \sum_{i=1}^t i^2$$

$$\|\bar{x}_t\|^2 = t - t + \frac{1}{(t+1)^2} \sum_{i=1}^t i^2$$

$$\|\bar{x}_t\|^2 = \frac{1}{(t+1)^2} \sum_{i=1}^t i^2$$

$$\|\bar{x}_t\|^2 \leq \frac{1}{(t+1)^2} \cdot \frac{(t+1)^3}{3}$$

$$\|\bar{x}_t\|^2 \leq \frac{t+1}{3}$$

$$\|\bar{x}_t\|^2 \leq \frac{1}{3}(t+1)$$

3. Fix indices $1 < i < j < n$ and a point $x \in \mathbb{R}^i \times \{0\}^{n-i}$. Show that equality $f_i(x) = f_j(x)$ holds and that the gradient $\nabla f_t(x)$ lies in $\mathbb{R}^{i+1} \times \{0\}^{n-(i+1)}$.

For $x \in \mathbb{R}^i \times \{0\}^{n-i}$, we can write $x = (x_1, \dots, x_i, 0, \dots, 0)$.

Plugging this x into the definitions of f_i and f_j , we observe that all terms involving coordinates beyond x_i vanish. So,

$$\begin{aligned} f_i(x) &= \frac{\beta}{4} \left(\frac{1}{2} \left(x_1^2 + \sum_{k=1}^{i-1} (x_k - x_{k+1})^2 + x_i^2 \right) - x_1 \right) \\ f_j(x) &= \frac{\beta}{4} \left(\frac{1}{2} \left(x_1^2 + \sum_{k=1}^{i-1} (x_k - x_{k+1})^2 + (x_i - 0)^2 + \sum_{k=i+1}^{j-1} (0 - 0)^2 + 0^2 \right) - x_1 \right) \\ f_j(x) &= \frac{\beta}{4} \left(\frac{1}{2} \left(x_1^2 + \sum_{k=1}^{i-1} (x_k - x_{k+1})^2 + x_i^2 \right) - x_1 \right) \end{aligned}$$

We see that $f_i(x) = f_j(x)$, where $x \in \mathbb{R}^i \times \{0\}^{n-i}$.

Let's now work with $\nabla f_t(x)$ for $x \in \mathbb{R}^i \times \{0\}^{n-i}$. We know that $x_{i+1} = 0$ all the way to $x_j = 0$. For x_{i+1} , the only nonzero term is $(x_i - x_{i+1})^2$. For the rest of x_j , where $j > i$, we have only zero terms: x_{i+2} all the way to x_n only have zero terms, no nonzero terms. The only possible nonzero gradient entries are in the first $i + 1$ coordinates and the zero gradient entries are in $n - (i + 1)$. This means $\nabla f_t(x)$ lies in $\mathbb{R}^{i+1} \times \{0\}^{n-(i+1)}$.

Extra Credit: After completing the exercise, I asked ChatGPT to do the question and this is what it stated:

For $j = t$, the partial derivative is:

$$\begin{aligned} \frac{\partial f_t}{\partial z_t} &= \frac{\beta}{8} (2(z_t - z_{t-1}) + 2z_t) \\ &= \frac{\beta}{8} (2z_t - 2z_{t-1} + 2z_t) \\ &= \frac{\beta}{8} (4z_t + 2z_{t-1}) \\ &= \frac{\beta}{4} (2z_t + z_{t-1}) \end{aligned}$$

Here, ChatGPT makes an error by ultimately stating $\frac{\partial f_t}{\partial z_t} = \frac{\beta}{4} (2z_t + z_{t-1})$, which is untrue. Rather, $\frac{\partial f_t}{\partial z_t} = \frac{\beta}{4} (2z_t - z_{t-1})$.

2. **Exercise 5.12:** In this exercise, you will consider the ridge regression problem:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - y\|_2^2 + \frac{\lambda}{2} \|x\|_2^2$$

which aims to recover a point x_{opt} from noisy linear measurements y . For $n = 100$, $m = 80$, and $\lambda = 1$, generate data as follows:

- the underlying signal is drawn $x_{\text{opt}} \sim N(0, I)$
- the measurement matrix $A \in \mathbb{R}^{m \times n}$ is drawn with independent standard Gaussian rows $A_i \sim N(0, I)$
- the observed data $y \in \mathbb{R}^m$ is $y = Ax_{\text{opt}} + \epsilon$ and $\epsilon \sim N(0, 0.25)$ independent and identically distributed

Note that the objective function is β smooth with $\beta = \|A^*A\|_{\text{op}} + \lambda$ and α strongly convex with $\alpha = \lambda$. In your experiments, you may set $\beta = 4(m + n) + \lambda$. Write code that generates the problem data as above and implement the following algorithms:

1. Gradient descent
2. Accelerated gradient descent

For each algorithm, plot the function value over the first 50 iterations on a semilog plot.

```

In [18]: import numpy as np
import scipy
import cvxpy as cp
import matplotlib.pyplot as plt
import math

def ridge_objective(x, A, y, lambda):
    return 0.5 * np.linalg.norm(A @ x - y)**2 + 0.5 * lambda * np.linalg.norm(x)**2

def gradient_descent(A, y, lambda, beta, num_iters=50):
    n = A.shape[1]
    x = np.zeros(n)
    f_values = []
    alpha = 1 / beta #Step size

    for k in range(num_iters):
        grad = A.T @ (A @ x - y) + lambda * x
        x = x - alpha * grad
        f_values.append(ridge_objective(x, A, y, lambda))
    return f_values

def accelerated_gradient_descent(A, y, lambda, beta, num_iters=50):
    n = A.shape[1]
    x = np.zeros(n)
    z = np.zeros(n)
    f_values = []
    alpha = 1 / beta #Step size
    gamma = 0
    for k in range(num_iters):
        prev_x = x.copy()
        grad = A.T @ (A @ z - y) + lambda * z
        x = z - alpha * grad
        if k == 0:
            gamma = 0
        else:
            gamma = (np.sqrt(beta) - np.sqrt(lambda)) / (np.sqrt(beta) + np.sqrt(lambda))
        z = x + gamma * (x - prev_x)
        f_values.append(ridge_objective(x, A, y, lambda))
    return f_values

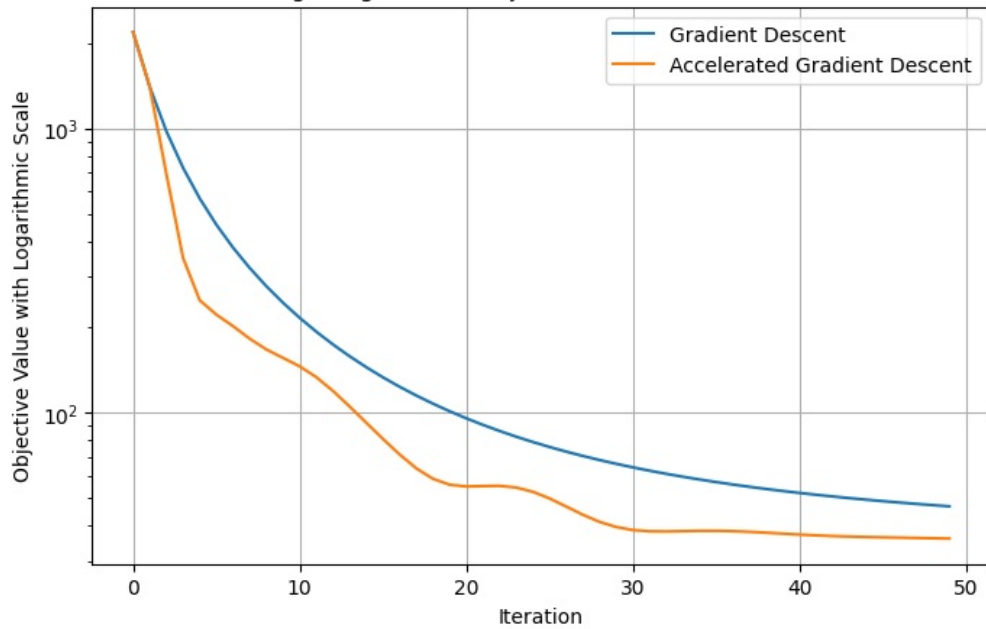
for i in range(5): #For loop here to show that answers vary
    n = 100
    m = 80
    lambda = 1
    x_opt = np.random.randn(n)
    A = np.random.randn(m, n)
    myVariance = 0.25
    mySTD = math.sqrt(myVariance)
    epsilon = np.random.normal(0, mySTD, m)
    y = A @ x_opt + epsilon
    beta = 4*(m+n) + lambda

    gd_values = gradient_descent(A, y, lambda, beta, num_iters=50)
    agd_values = accelerated_gradient_descent(A, y, lambda, beta, num_iters=50)

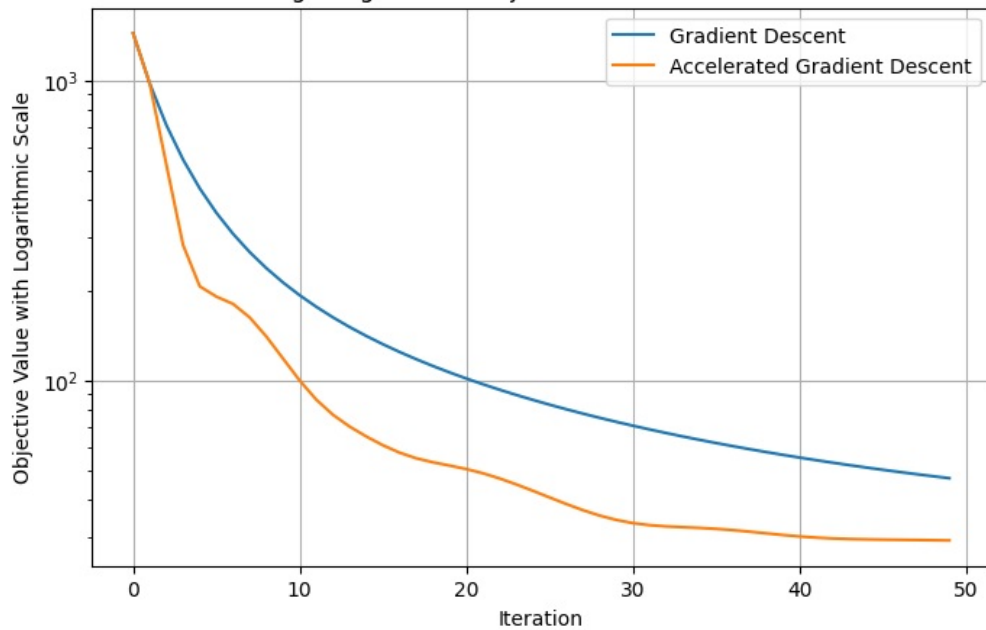
    plt.figure(figsize=(8, 5))
    plt.semilogy(gd_values, label = "Gradient Descent")
    plt.semilogy(agd_values, label = "Accelerated Gradient Descent")
    plt.xlabel("Iteration")
    plt.ylabel("Objective Value with Logarithmic Scale")
    plt.title("Ridge Regression: Objective Value vs Iteration "+str(i+1))
    plt.legend()
    plt.grid(True)
    plt.show()

```

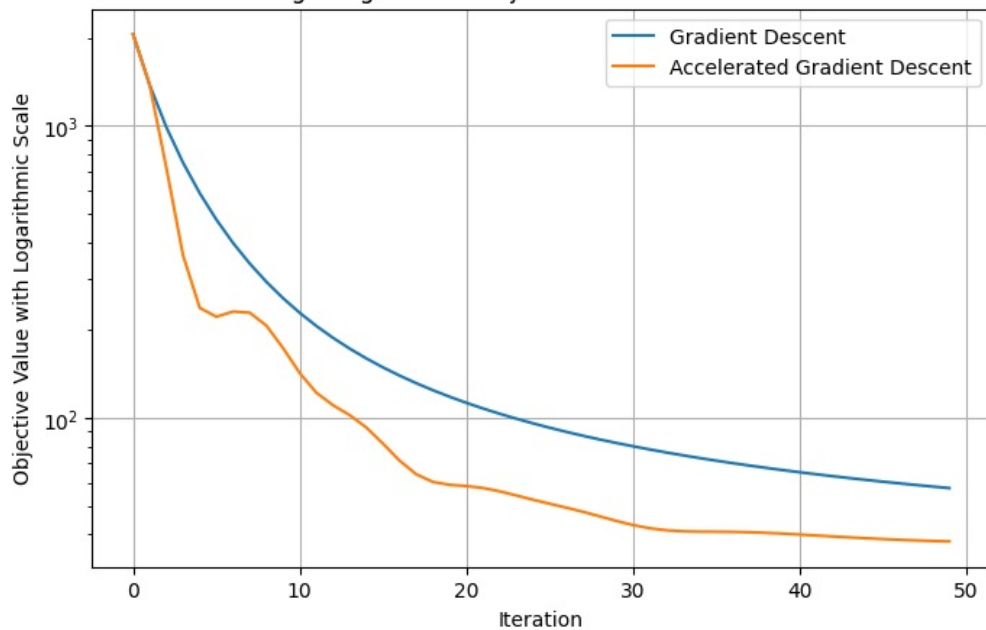
Ridge Regression: Objective Value vs Iteration 1



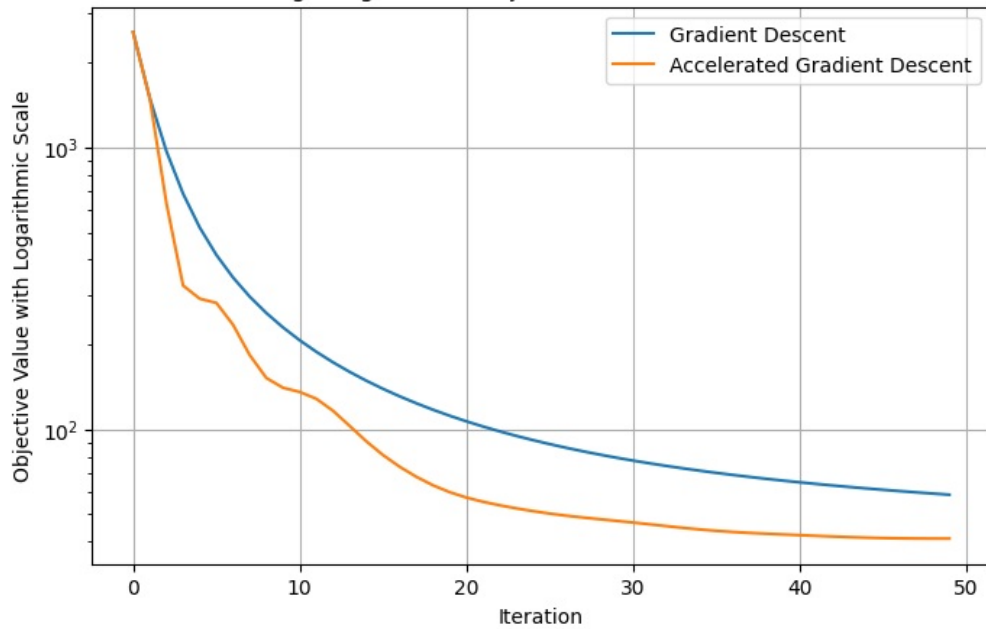
Ridge Regression: Objective Value vs Iteration 2



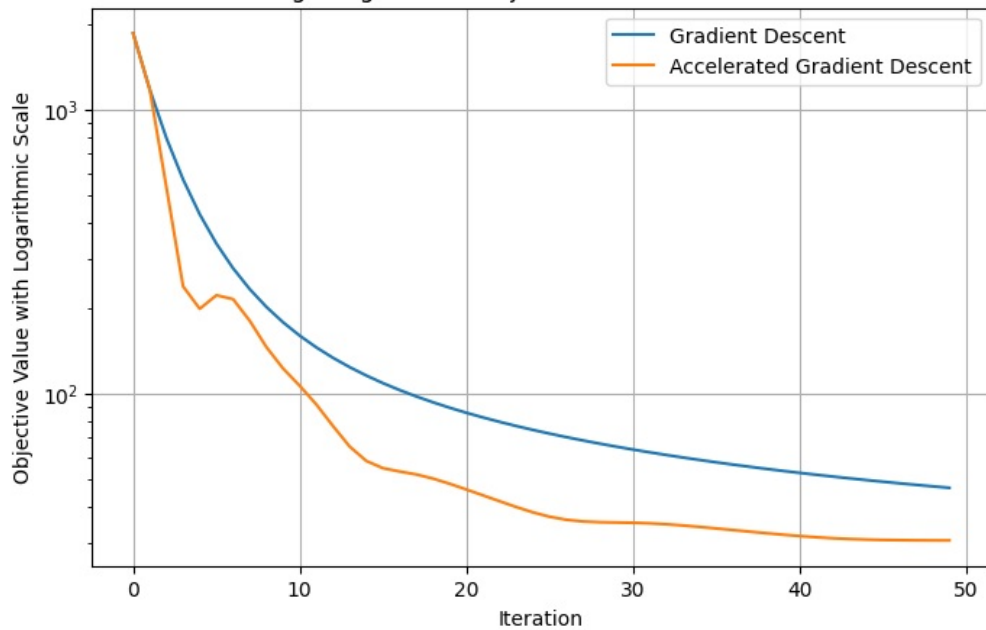
Ridge Regression: Objective Value vs Iteration 3



Ridge Regression: Objective Value vs Iteration 4



Ridge Regression: Objective Value vs Iteration 5



Note: I had to convert to HTML first and then convert to PDF as my computer wouldn't directly convert from Python to PDF.

Extra Credit: After completing the exercise, I asked ChatGPT to do the question and this is what it stated:

```
def accelerated_gradient_descent(A, y, lambda, beta, num_iters=50):
    n = A.shape[1]
    x = np.zeros(n)
    z = np.zeros(n)
    f_values = []
    alpha = 1 / beta # Step size
    gamma = 0
    for k in range(num_iters):
        prev_x = x.copy()
        grad = A.T @ (A @ z - y) + lambda * z
        x = z - alpha * grad
        if k == 0:
            gamma = 0
        else:
            gamma = (np.sqrt(lambda) - np.sqrt(beta)) / (np.sqrt(lambda) + np.sqrt(beta))
        z = x + gamma * (x - prev_x)
        f_values.append(ridge_objective(x, A, y, lambda))
    return f_values
```

Here, the error is that ChatGPT said $\gamma = (\sqrt{\lambda} - \sqrt{\beta}) / (\sqrt{\lambda} + \sqrt{\beta})$ which is incorrect. Rather, $\gamma = (\sqrt{\beta} - \sqrt{\lambda}) / (\sqrt{\beta} + \sqrt{\lambda})$.

3. **Exercise 5.13:** Consider the huberized version of Ridge Regression:

$$\min_{x \in \mathbb{R}^n} \sum_{i=1}^m h_{\eta}(y_i - a_i^T x) + \frac{\lambda}{2} \|x\|_2^2$$

where a_i^T are the rows of A , and h_{η} is the huber function with parameter η :

$$h_{\eta} = \begin{cases} \frac{1}{2}w^2, & \text{if } |w| \leq \eta \\ \eta(|w| - \frac{\eta}{2}), & \text{otherwise} \end{cases}$$

The huber function penalizes large deviations less than the quadratic cost function, and is therefore often used when outliers are present in the data. Note that the objective function in this case is β smooth with $\beta > m\eta + \lambda$ and α strongly convex with $\alpha = \lambda$.

Generate x_{opt} and A as in **Exercise 5.12** and generate $y = Ax_{\text{opt}} + \epsilon$, where ϵ contains 5 outliers drawn from $N(0, 25)$. Implement the same two algorithms as in **Exercise 5.12** on this problem, and plot the function values for the first 100 iterations in a semilog plot.

```

In [13]: import numpy as np
import matplotlib.pyplot as plt
import math

def huber_loss(w, eta):
    return np.where(np.abs(w) <= eta, 0.5 * w**2, eta * (np.abs(w) - 0.5 * eta))

def huber_grad(w, eta):
    return np.where(np.abs(w) <= eta, w, eta * np.sign(w))

def huberized_objective(x, A, y, lambda, eta):
    w = y - A @ x
    return np.sum(huber_loss(w, eta)) + 0.5 * lambda * np.linalg.norm(x)**2

def huberized_gradient(x, A, y, lambda, eta):
    w = y - A @ x
    grad_loss = -A.T @ huber_grad(w, eta)
    grad_reg = lambda * x
    return grad_loss + grad_reg

def gradient_descent(A, y, lambda, eta, beta, num_iters=50):
    n = A.shape[1]
    x = np.zeros(n)
    f_values = []
    alpha = 1 / beta #Step size

    for k in range(num_iters):
        grad = huberized_gradient(x, A, y, lambda, eta)
        x = x - alpha * grad
        f_values.append(huberized_objective(x, A, y, lambda, eta))
    return f_values

def accelerated_gradient_descent(A, y, lambda, eta, beta, num_iters=50):
    n = A.shape[1]
    x = np.zeros(n)
    z = np.zeros(n)
    f_values = []
    alpha = 1 / beta #Step size
    gamma = 0
    for k in range(num_iters):
        prev_x = x.copy()
        grad = huberized_gradient(z, A, y, lambda, eta)
        x = z - alpha * grad
        if k == 0:
            gamma = 0
        else:
            gamma = (np.sqrt(beta) - np.sqrt(lambda)) / (np.sqrt(beta) + np.sqrt(lambda))
        z = x + gamma * (x - prev_x)
        f_values.append(huberized_objective(x, A, y, lambda, eta))
    return f_values

n = 100
m = 80
lambda = 1

beta = 4*(m+n) + lambda

```

```

In [14]: eta = 1.0

for i in range(5):
    x_opt = np.random.randn(n)
    A = np.random.randn(m, n)
    myVariance = 0.25
    mySTD = math.sqrt(myVariance)
    epsilon = np.random.normal(0, mySTD, m)

    outlier_indices = np.random.choice(m, size=5, replace=False)
    mu = 0
    outliers_drawn = 25
    kempSTD = math.sqrt(outliers_drawn)
    epsilon[outlier_indices] = np.random.normal(mu, kempSTD, 5)
    y = A @ x_opt + epsilon

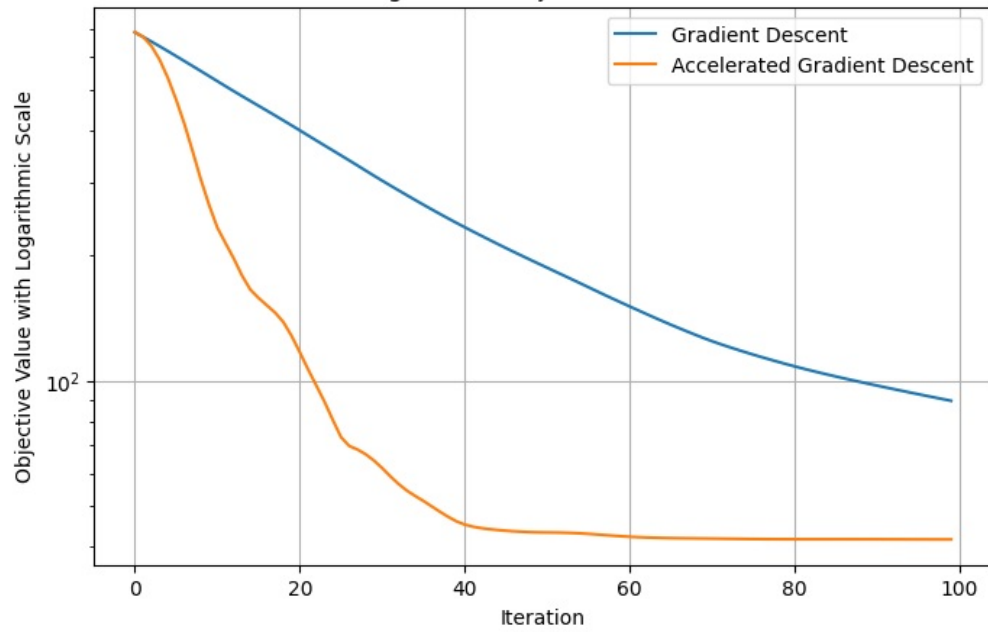
    gd_values = gradient_descent(A, y, lambda, eta, beta, num_iters=100)
    agd_values = accelerated_gradient_descent(A, y, lambda, eta, beta, num_iters=100)

    plt.figure(figsize=(8, 5))
    plt.semilogy(gd_values, label = "Gradient Descent")
    plt.semilogy(agd_values, label = "Accelerated Gradient Descent")
    plt.xlabel("Iteration")
    plt.ylabel("Objective Value with Logarithmic Scale")

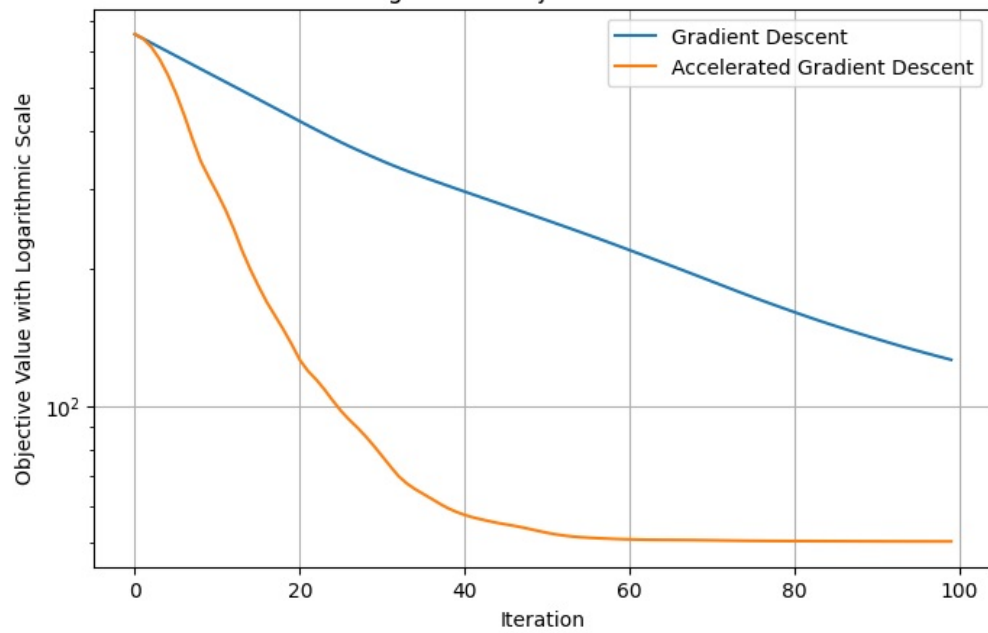
```

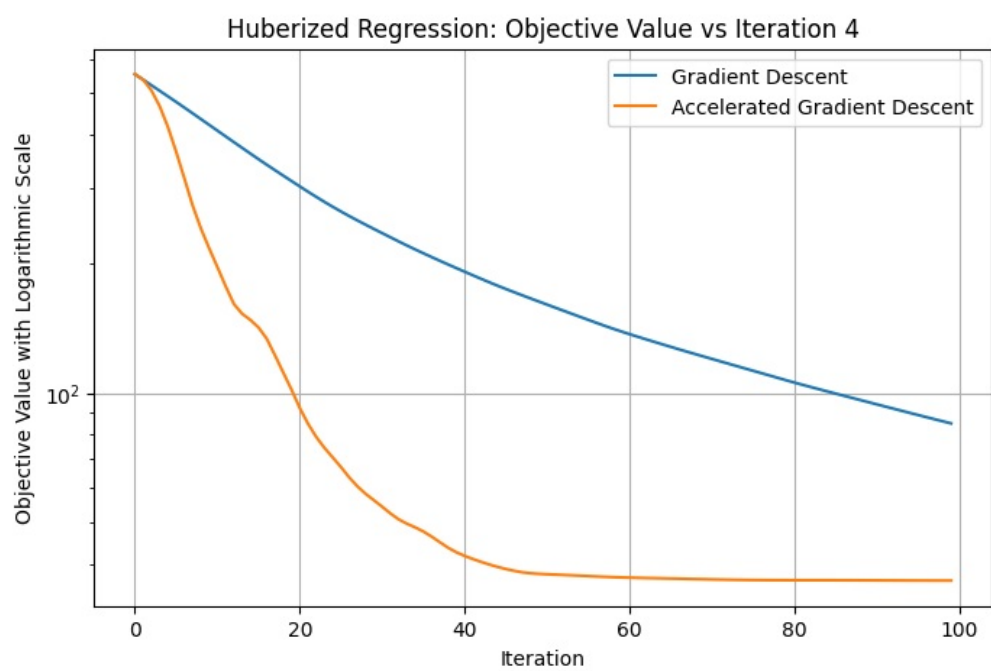
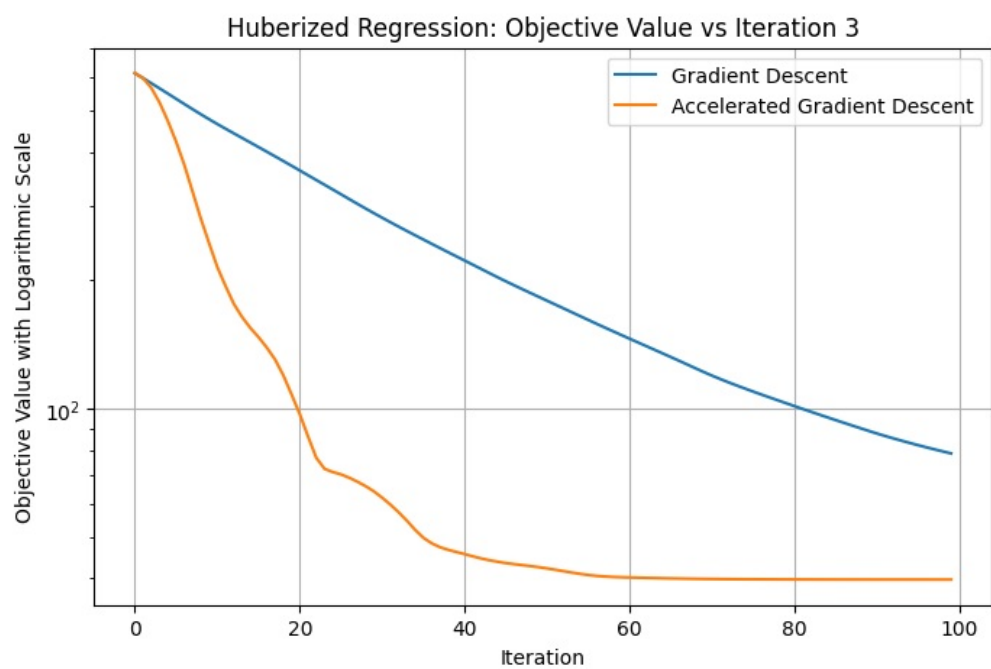
```
plt.title("Huberized Regression: Objective Value vs Iteration "+str(i+1))
plt.legend()
plt.grid(True)
plt.show()
```

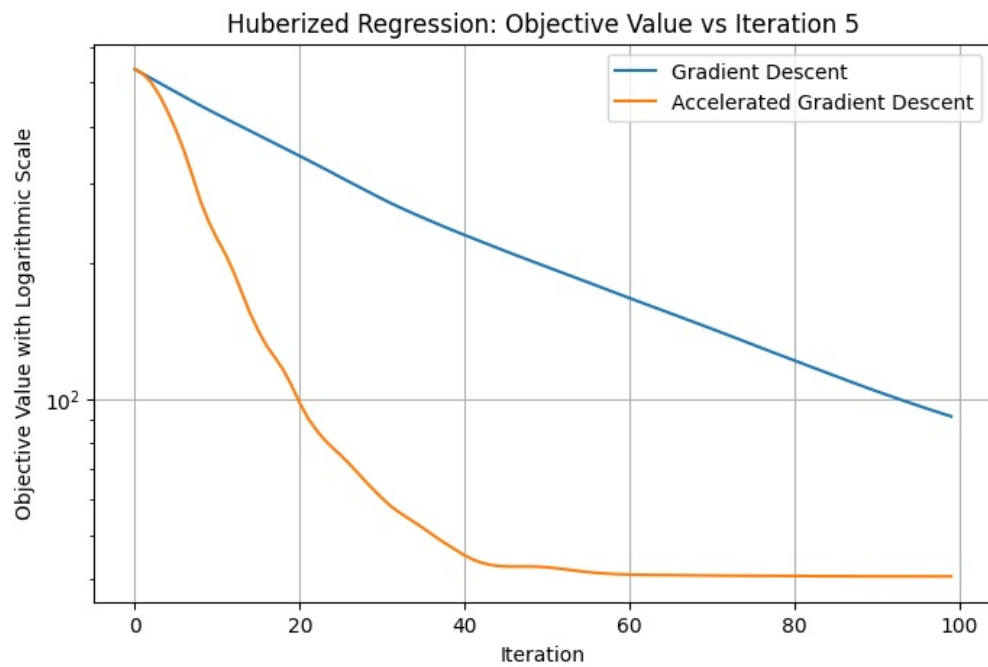
Huberized Regression: Objective Value vs Iteration 1



Huberized Regression: Objective Value vs Iteration 2







```
In [15]: eta = 2.0

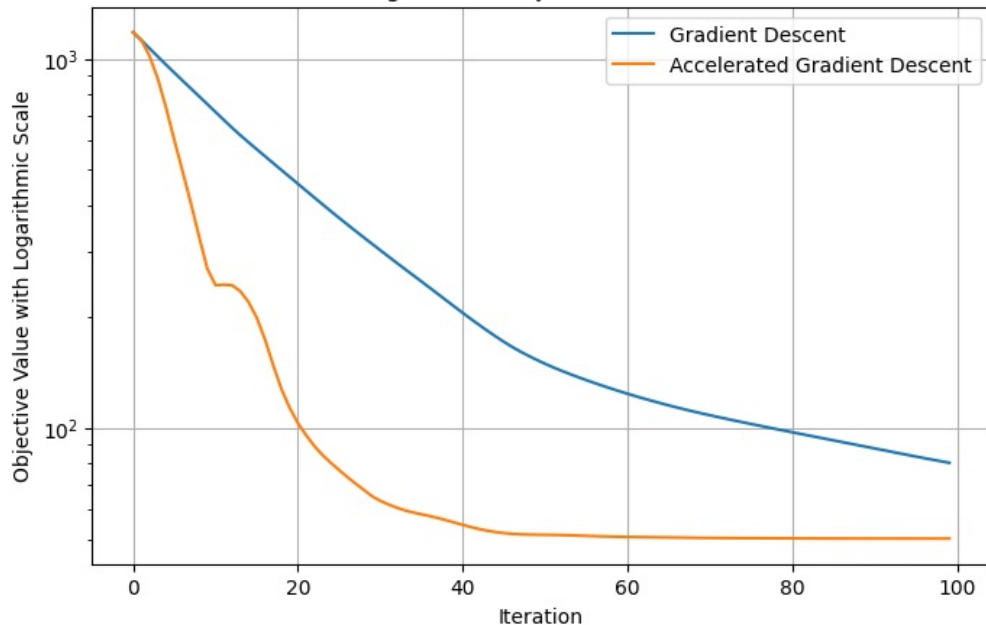
for i in range(5):
    x_opt = np.random.randn(n)
    A = np.random.randn(m, n)
    myVariance = 0.25
    mySTD = math.sqrt(myVariance)
    epsilon = np.random.normal(0, mySTD, m)

    outlier_indices = np.random.choice(m, size=5, replace=False)
    mu = 0
    outliers_drawn = 25
    kempSTD = math.sqrt(outliers_drawn)
    epsilon[outlier_indices] = np.random.normal(mu, kempSTD, 5)
    y = A @ x_opt + epsilon

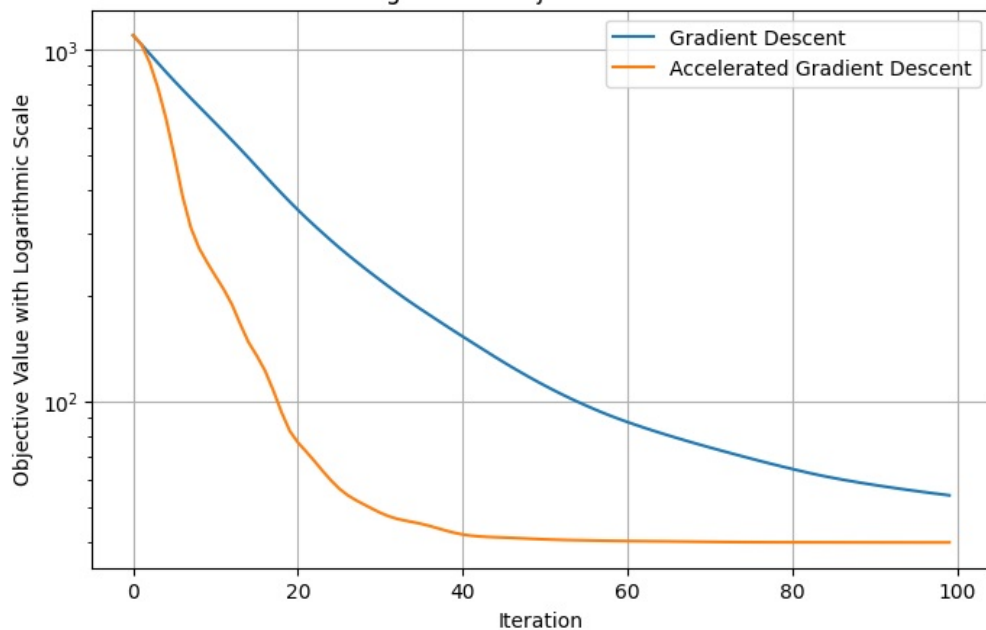
    gd_values = gradient_descent(A, y, lambda, eta, beta, num_iters=100)
    agd_values = accelerated_gradient_descent(A, y, lambda, eta, beta, num_iters=100)

    plt.figure(figsize=(8, 5))
    plt.semilogy(gd_values, label = "Gradient Descent")
    plt.semilogy(agd_values, label = "Accelerated Gradient Descent")
    plt.xlabel("Iteration")
    plt.ylabel("Objective Value with Logarithmic Scale")
    plt.title("Huberized Regression: Objective Value vs Iteration "+str(i+1))
    plt.legend()
    plt.grid(True)
    plt.show()
```

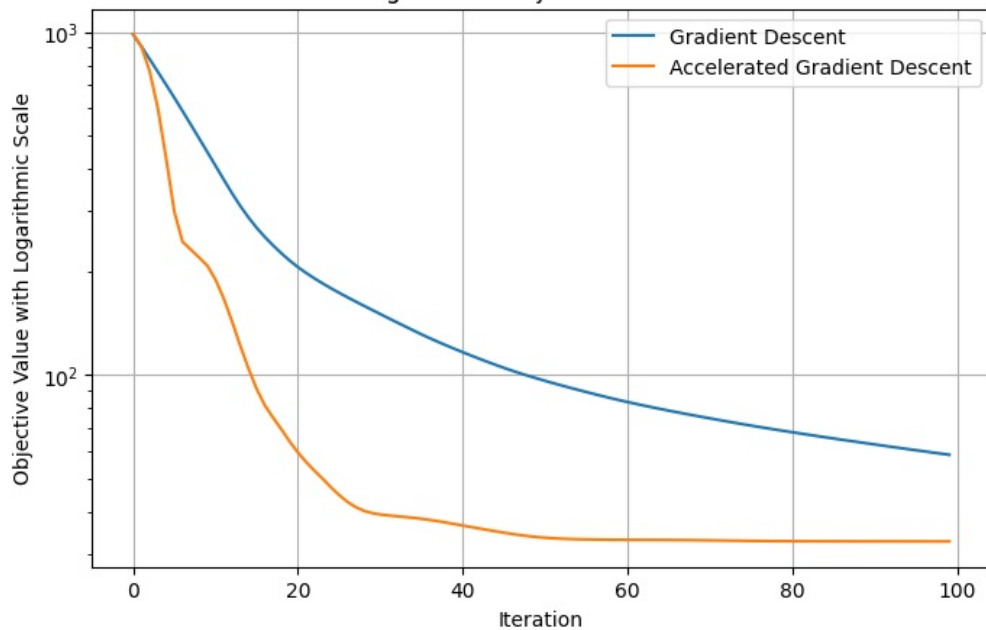
Huberized Regression: Objective Value vs Iteration 1

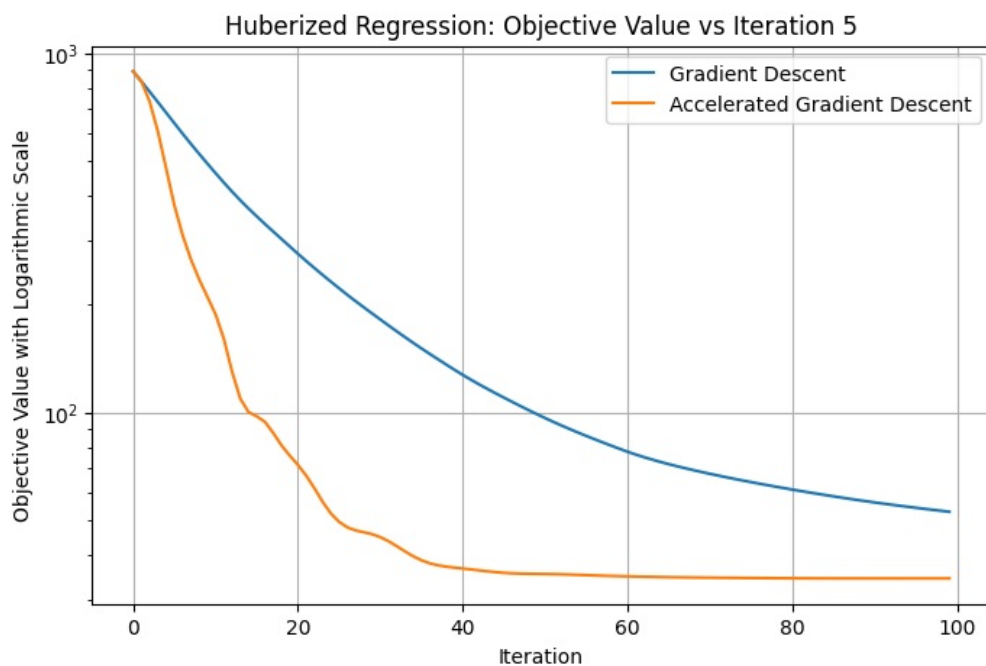
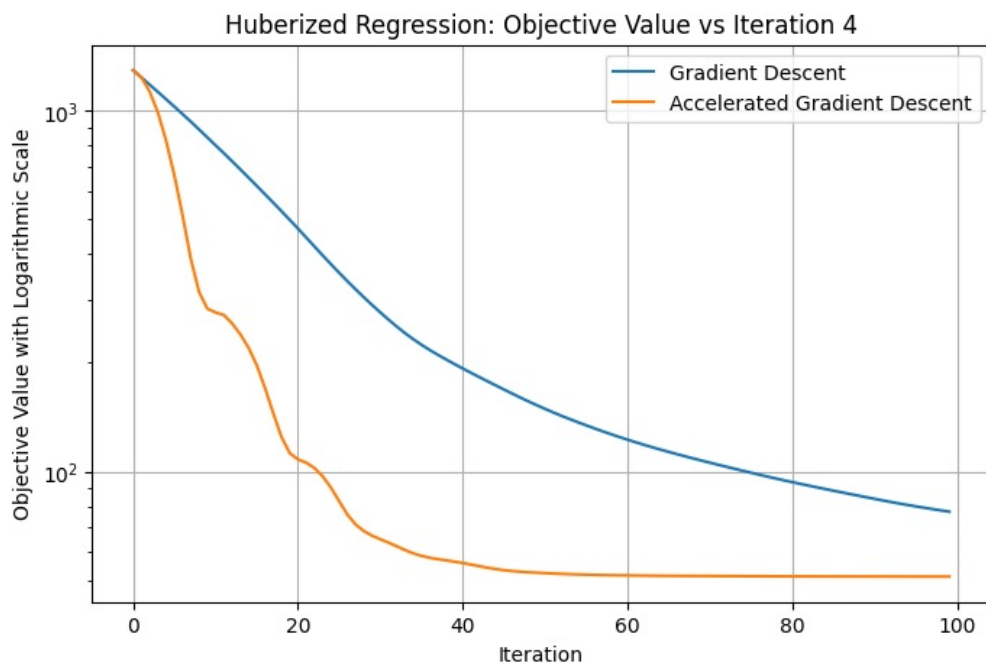


Huberized Regression: Objective Value vs Iteration 2



Huberized Regression: Objective Value vs Iteration 3





```
In [16]: eta = 5.0

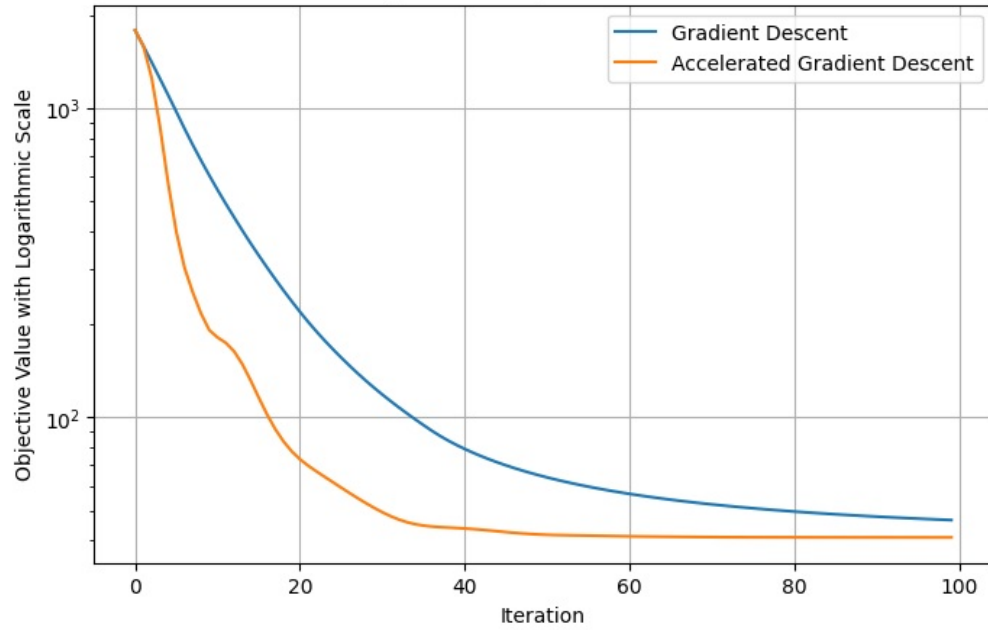
for i in range(5):
    x_opt = np.random.randn(n)
    A = np.random.randn(m, n)
    myVariance = 0.25
    mySTD = math.sqrt(myVariance)
    epsilon = np.random.normal(0, mySTD, m)

    outlier_indices = np.random.choice(m, size=5, replace=False)
    mu = 0
    outliers_drawn = 25
    kempSTD = math.sqrt(outliers_drawn)
    epsilon[outlier_indices] = np.random.normal(mu, kempSTD, 5)
    y = A @ x_opt + epsilon

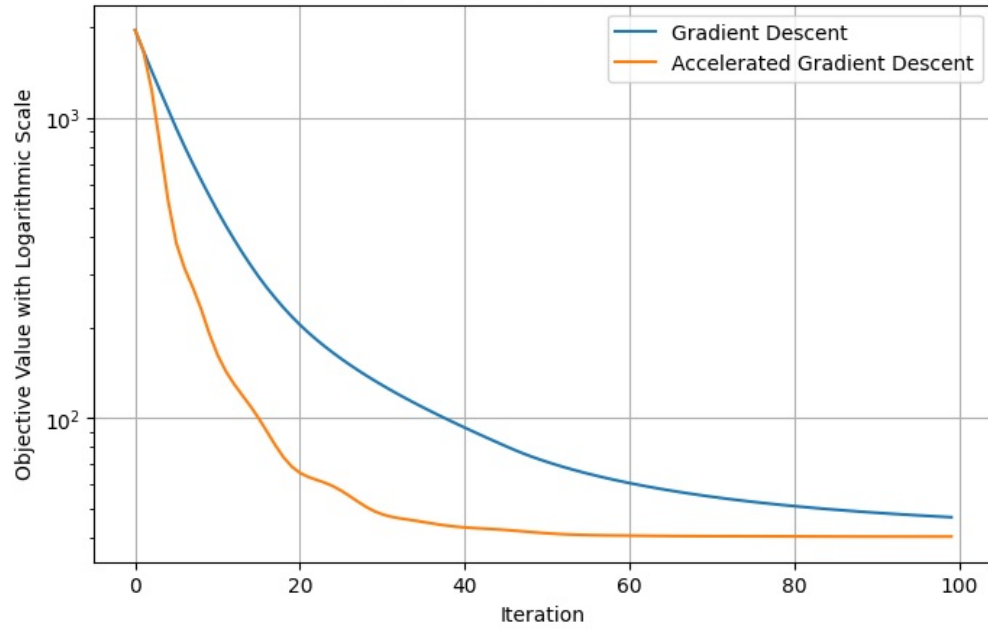
    gd_values = gradient_descent(A, y, lambda, eta, beta, num_iters=100)
    agd_values = accelerated_gradient_descent(A, y, lambda, eta, beta, num_iters=100)

    plt.figure(figsize=(8, 5))
    plt.semilogy(gd_values, label = "Gradient Descent")
    plt.semilogy(agd_values, label = "Accelerated Gradient Descent")
    plt.xlabel("Iteration")
    plt.ylabel("Objective Value with Logarithmic Scale")
    plt.title("Huberized Regression: Objective Value vs Iteration "+str(i+1))
    plt.legend()
    plt.grid(True)
```

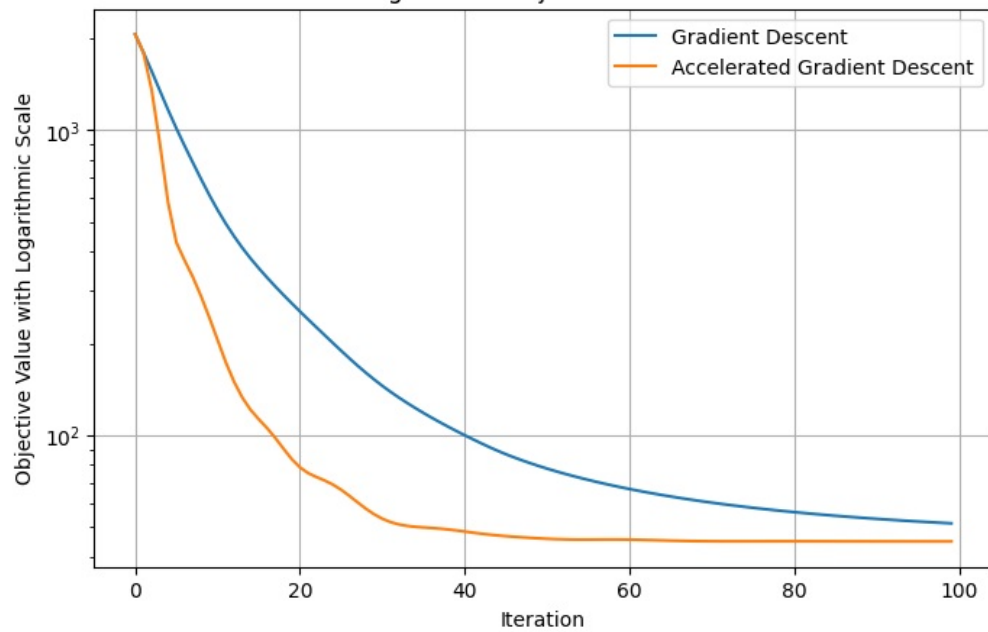

Huberized Regression: Objective Value vs Iteration 1



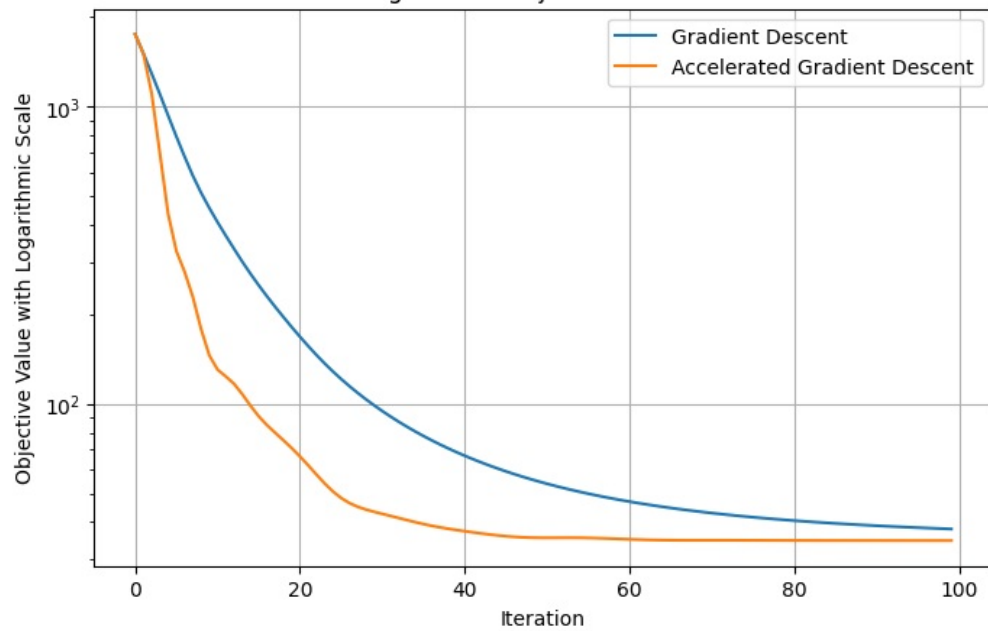
Huberized Regression: Objective Value vs Iteration 2

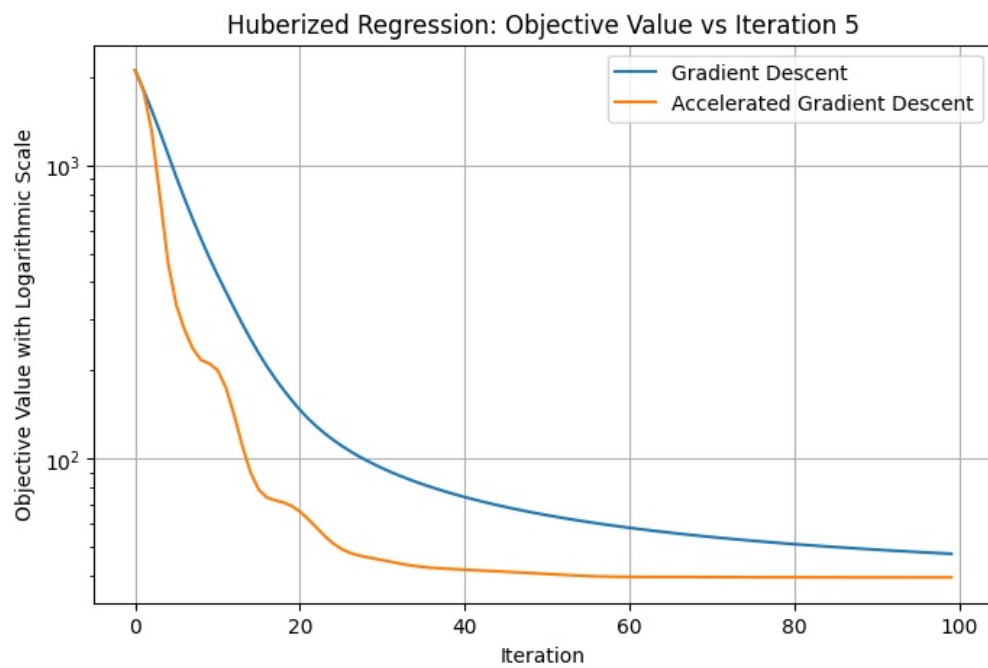


Huberized Regression: Objective Value vs Iteration 3



Huberized Regression: Objective Value vs Iteration 4





I have changed η through each of my cell blocks because I wanted to experiment with different η values as the Professor didn't specify which η value we had to use.

Processing math: 100%

Note: I had to convert to HTML first and then convert to PDF as my computer wouldn't directly convert from Python to PDF.

Extra Credit: After completing the exercise, I asked ChatGPT to do the question and this is what it stated:

python

```
def huberized_gradient(x, A, y, lambda, eta):  
    w = y - A @ x  
    grad_loss = -A.T @ huber_grad(w, eta)  
    grad_reg = -lambda * x  
    return grad_loss + grad_reg
```

Here, the error is that ChatGPT said `grad_reg = -lambda * x` which is incorrect. Rather, `grad_reg = lambda * x`.

4. **Exercise 5.14:** Logistic Regression: Consider the regularized logistic regression problem:

$$\min_{\theta \in \mathbb{R}^n} \sum_{i=1}^m \ln(1 + e^{-y_i \theta^T x_i}) + \frac{\lambda}{2} \|\theta\|_2^2$$

Here, $y_i \in \{\pm 1\}$ are binary labels for the data points $x_i \in \mathbb{R}^n$ and we are trying to find the best vector θ of parameters for the model

$$P(y = 1|x, \theta) = \frac{1}{1 + e^{-\theta^T x}}$$

We are doing this by minimizing the negative log likelihood function plus a regularization term. Implement the same two algorithms as before, with $n = 50$ and $m = 100$ and $\lambda = 1$. Generate data as follows:

1. $\theta_{\text{opt}} = (1, \dots, 1)^T$
2. $X \in \mathbb{R}^{m \times n}$ has independent and identically distributed standard normal entries with rows x_i^T
3. $z = X\theta_{\text{opt}}$ and the true vector of probabilities is $p = \frac{1}{1+e^{-z}}$, where the operations are applied entrywise to the vector z
4. $y \in \{\pm 1\}^m$ has independent Bernoulli entries with $P(y_i = 1) = p_i$
5. θ_0 has independent and identically distributed standard normal entries

Implement the same two algorithms as in **Exercise 5.12** on this problem, and plot the function values for the first 100 iterations in a semilog plot. You may take the smoothness parameter of the objective function to be $\beta = 100$ and the strong convexity parameter to be $\alpha = 1$.

```

In [33]: import numpy as np
import matplotlib.pyplot as plt
import scipy

def sigmoid(z):
    return 1/(1+np.exp(-z))

def logistic_objective(theta, X, y, lambda):
    yz = y * (X @ theta)
    my_loss = scipy.special.log1p(1 + np.exp(-yz)).sum()
    r = 0.5 * lambda * np.linalg.norm(theta)**2
    return my_loss + r

def logistic_gradient(theta, X, y, lambda):
    yz = y * (X @ theta)
    ja = 1 / (1+np.exp(yz))
    toRETURN = -((y * ja) @ X) + lambda * theta
    return toRETURN

def gradient_descent(X, y, lambda, beta, num_iters=100):
    n = X.shape[1]
    theta = np.random.randn(n)
    f_values = []
    alpha = 1 / beta
    for k in range(num_iters):
        g = logistic_gradient(theta, X, y, lambda)
        theta = theta - alpha * g
        f_values.append(logistic_objective(theta, X, y, lambda))
    return f_values

def accelerated_gradient_descent(X, y, lambda, beta, num_iters=100):
    n = X.shape[1]
    theta = np.random.randn(n)
    yk = theta.copy()
    f_values = []
    alpha = 1 / beta
    jf = 1 # Used for strong convexity
    for k in range(num_iters):
        previous_theta = theta.copy()
        gr = logistic_gradient(yk, X, y, lambda)
        theta = yk - alpha * gr
        if k == 0:
            gamma = 0
        else:
            gamma = (np.sqrt(beta) - np.sqrt(jf)) / (np.sqrt(beta) + np.sqrt(jf))
        yk = theta + gamma * (theta - previous_theta)
        f_values.append(logistic_objective(theta, X, y, lambda))
    return f_values

for i in range(5): #For loop here to show that answers vary
    n = 50
    m = 100
    lambda = 1
    beta = 100

    theta_opt = np.ones(n)
    X = np.random.randn(m, n)
    z = X @ theta_opt
    p1857 = 1 / (1+np.exp(-z))

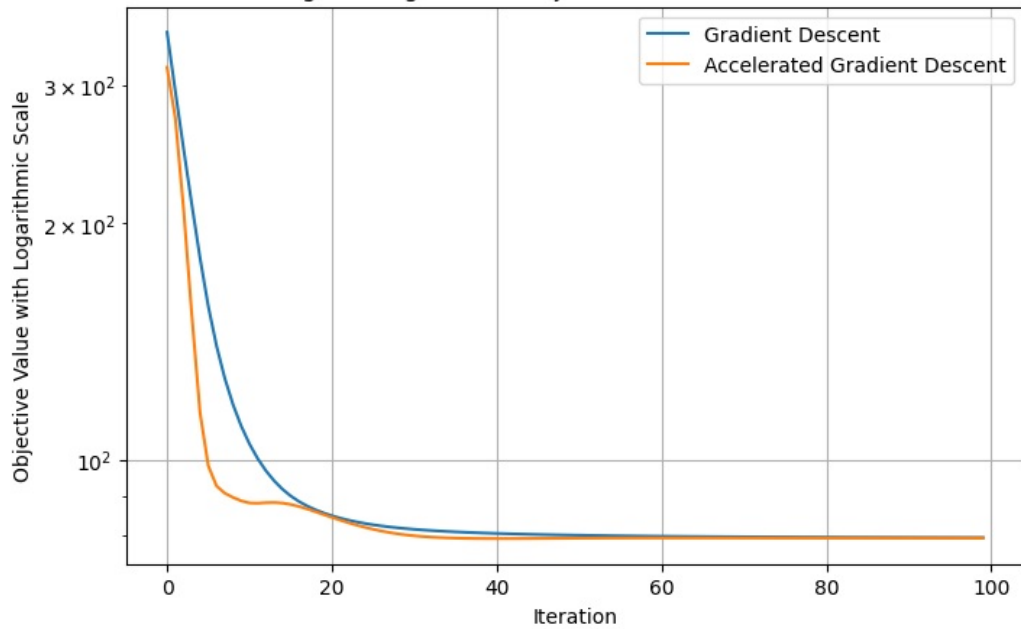
    y = np.random.binomial(1, p1857)
    y = 2*y-1

    gd_values = gradient_descent(X, y, lambda, beta, num_iters=100)
    agd_values = accelerated_gradient_descent(X, y, lambda, beta, num_iters=100)

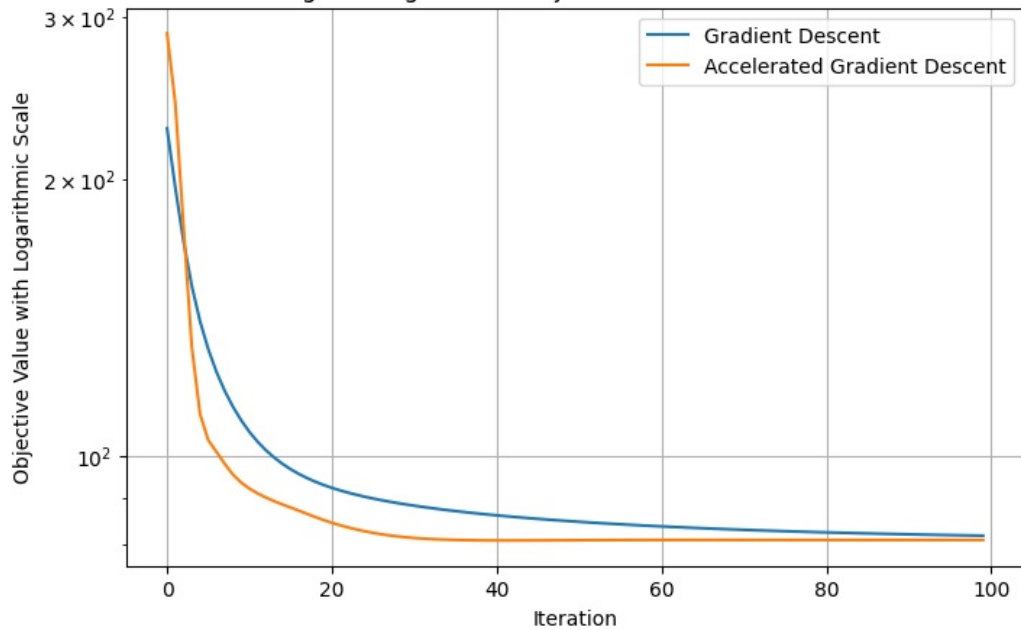
    plt.figure(figsize=(8, 5))
    plt.semilogy(gd_values, label="Gradient Descent")
    plt.semilogy(agd_values, label="Accelerated Gradient Descent")
    plt.xlabel("Iteration")
    plt.ylabel("Objective Value with Logarithmic Scale")
    plt.title("Logistic Regression: Objective Value vs Iteration "+str(i+1))
    plt.legend()
    plt.grid(True)
    plt.show()

```

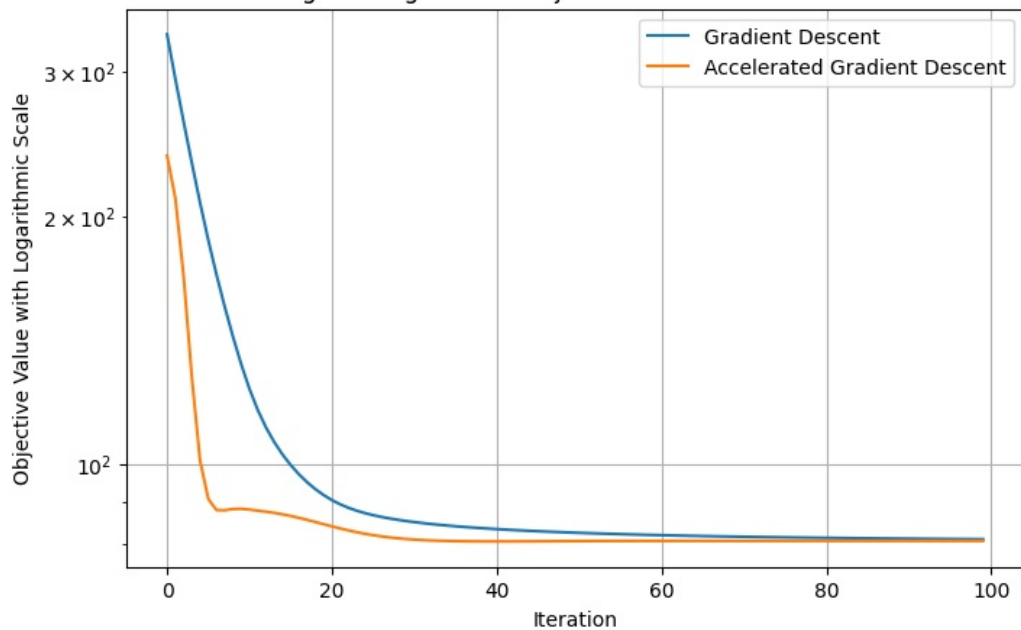
Logistic Regression: Objective Value vs Iteration 1

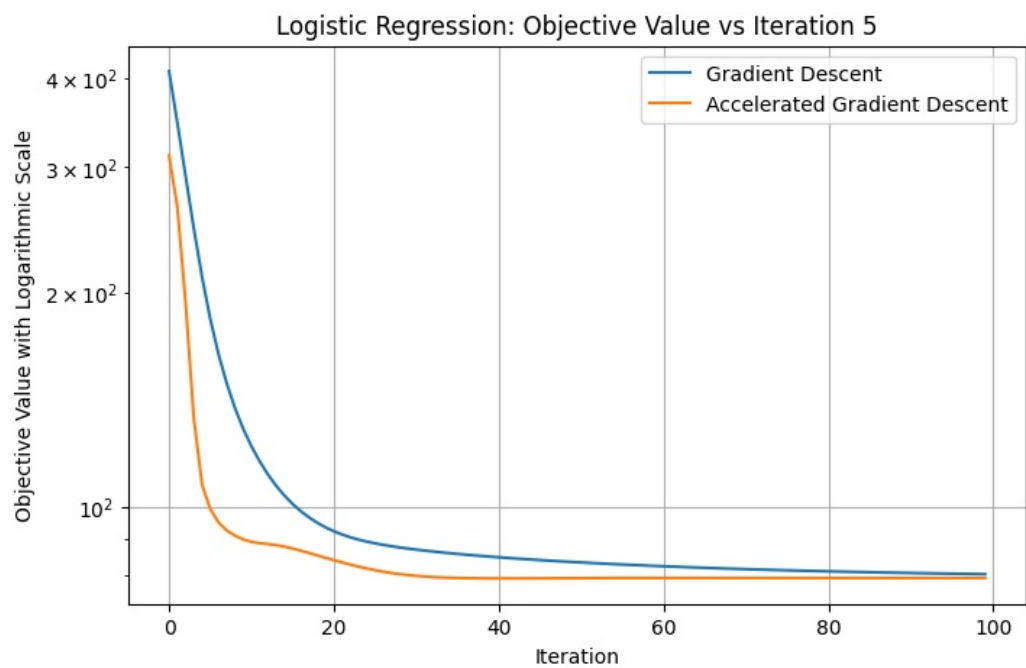
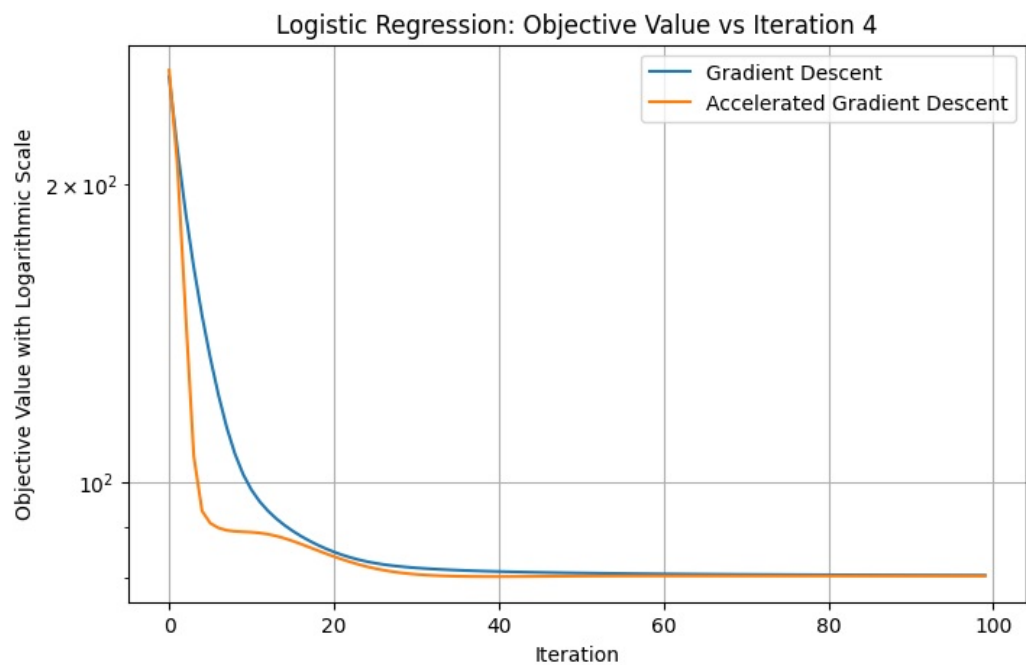


Logistic Regression: Objective Value vs Iteration 2



Logistic Regression: Objective Value vs Iteration 3





Acknowledgment: For this question, I got help from Collin.

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

Note: I had to convert to HTML first and then convert to PDF as my computer wouldn't directly convert from Python to PDF.

Extra Credit: After completing the exercise, I asked ChatGPT to do the question and this is what it stated:

```
python Copy Edit  
  
def accelerated_gradient_descent(X, y, lmbda, beta, num_iters=100):  
    n = X.shape[1]  
    theta = np.random.randn(n)  
    yk = theta.copy()  
    f_values = []  
    alpha = 1 / beta  
    gamma = 0  
    for k in range(num_iters):  
        previous_theta = theta.copy()  
        gr = logistic_gradient(yk, X, y, lmbda)  
        theta = yk - alpha * gr  
        if k == 0:  
            gamma = 0  
        else:  
            gamma = (np.sqrt(beta) - np.sqrt(lmbda)) / (np.sqrt(beta) + np.sqrt(lmbda))  
        yk = theta - gamma * (theta - previous_theta)  
        f_values.append(logistic_objective(theta, X, y, lmbda))  
    return f_values
```

Here, the error is that ChatGPT said `yk = theta - gamma * (theta - previous_theta)` which is incorrect. Rather, `yk = theta + gamma * (theta - previous_theta)`.

5. **Exercise 5.15:** Polyak stepsize: Consider a differentiable convex function $f : \mathbb{E} \rightarrow \mathbb{R}$ and let x^* be any of its minimizers. Consider the gradient descent iterates

$$x_{k+1} = x_k - \gamma_k \nabla f(x_k)$$

for some sequence $\gamma_k \geq 0$.

1. By writing the term $\|x_{k+1} - x^*\|^2 = \|(x_{k+1} - x_k) + (x_k - x^*)\|^2$ and expanding the square, deduce the estimate

$$\frac{1}{2} \|x_{k+1} - x^*\|^2 \leq \frac{1}{2} \|x_k - x^*\|^2 - \gamma_k (f(x_k) - f(x^*)) + \frac{\gamma_k^2}{2} \|\nabla f(x_k)\|^2$$

$$\begin{aligned} \|x_{k+1} - x^*\|^2 &= \|(x_{k+1} - x_k) + (x_k - x^*)\|^2 \\ \|x_{k+1} - x^*\|^2 &= \|x_{k+1} - x_k\|^2 + \|x_k - x^*\|^2 + 2(x_{k+1} - x_k)^T (x_k - x^*) \\ x_{k+1} &= x_k - \gamma_k \nabla f(x_k) \\ x_{k+1} - x_k &= -\gamma_k \nabla f(x_k) \\ \|x_{k+1} - x^*\|^2 &= \|x_k - x^*\|^2 + \gamma_k^2 \|\nabla f(x_k)\|^2 - 2\gamma_k \nabla f(x_k)^T (x_k - x^*) \\ f(x^*) &\geq f(x_k) + \nabla f(x_k)^T (x^* - x_k) \quad (\text{By convexity}) \\ f(x^*) - f(x_k) &\geq \nabla f(x_k)^T (x^* - x_k) \\ f(x_k) - f(x^*) &\leq \nabla f(x_k)^T (x_k - x^*) \\ -2\gamma_k \nabla f(x_k)^T (x_k - x^*) &\leq -2\gamma_k (f(x_k) - f(x^*)) \\ \|x_{k+1} - x^*\|^2 &\leq \|x_k - x^*\|^2 - 2\gamma_k (f(x_k) - f(x^*)) + \gamma_k^2 \|\nabla f(x_k)\|^2 \quad (\text{Substituting Back}) \\ \frac{1}{2} \|x_{k+1} - x^*\|^2 &\leq \frac{1}{2} \|x_k - x^*\|^2 - \gamma_k (f(x_k) - f(x^*)) + \frac{\gamma_k^2}{2} \|\nabla f(x_k)\|^2 \end{aligned}$$

2. Supposing that you know the minimal value f^* of f , show that the sequence $\gamma_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2}$ minimizes the right hand side of

$$\frac{1}{2} \|x_{k+1} - x^*\|^2 \leq \frac{1}{2} \|x_k - x^*\|^2 - \gamma_k (f(x_k) - f(x^*)) + \frac{\gamma_k^2}{2} \|\nabla f(x_k)\|^2$$

in γ , thereby yielding the guarantee

$$\|x_{k+1} - x^*\|^2 \leq \|x_k - x^*\|^2 - \left(\frac{f(x_k) - f^*}{\|\nabla f(x_k)\|} \right)^2$$

Setting

$$\gamma_k = \arg \min_{\gamma \geq 0} \left\{ -\gamma (f(x_k) - f^*) + \frac{\gamma^2}{2} \|\nabla f(x_k)\|^2 \right\}$$

We define $g(\gamma)$ as so and minimize with respect to γ :

$$g(\gamma) = -\gamma (f(x_k) - f^*) + \frac{\gamma^2}{2} \|\nabla f(x_k)\|^2$$

Since this function is quadratic in γ and the quadratic coefficient $\frac{1}{2} \|\nabla f(x_k)\|^2$ is positive and we have $g(\gamma)$ as convex. The minimum can be found by setting its derivative to 0.

$$g'(\gamma) = -(f(x_k) - f^*) + \gamma \|\nabla f(x_k)\|^2$$

$$g'(\gamma) = 0 \quad (\text{Set the derivative equal to 0})$$

$$-(f(x_k) - f^*) + \gamma \|\nabla f(x_k)\|^2 = 0$$

$$\gamma \|\nabla f(x_k)\|^2 = f(x_k) - f^*$$

$$\gamma = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2}$$

Thus, we see that the sequence $\gamma_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2}$, where

$$\gamma_k = \arg \min_{\gamma \geq 0} \left\{ -\gamma(f(x_k) - f^*) + \frac{\gamma^2}{2} \|\nabla f(x_k)\|^2 \right\}$$

minimizes the right hand side

$$\frac{1}{2} \|x_{k+1} - x^*\|^2 \leq \frac{1}{2} \|x_k - x^*\|^2 - \gamma_k (f(x_k) - f(x^*)) + \frac{\gamma_k^2}{2} \|\nabla f(x_k)\|^2$$

3. Let x_k be the sequence generated by the gradient method with $\gamma_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2}$. Supposing that f is β smooth, conclude the estimate

$$f\left(\frac{1}{k} \sum_{i=0}^{k-1} x_i\right) - f^* \leq \frac{2\beta \|x_0 - x^*\|^2}{k}$$

If f is in addition α strongly convex, derive the guarantee

$$\|x_{k+1} - x^*\|^2 \leq \left(1 - \frac{\alpha}{4\beta}\right) \|x_k - x^*\|^2$$

Using $\gamma_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2}$, we have that

$$\frac{1}{2} \|x_{k+1} - x^*\|^2 \leq \frac{1}{2} \|x_k - x^*\|^2 - \frac{(f(x_k) - f^*)^2}{2\|\nabla f(x_k)\|^2}$$

So,

$$\|x_{k+1} - x^*\|^2 \leq \|x_k - x^*\|^2 - \frac{(f(x_k) - f^*)^2}{\|\nabla f(x_k)\|^2}$$

Supposing f is β smooth, we have:

$$\|\nabla f(x)\|^2 \leq 2\beta(f(x) - f^*)$$

$$\frac{(f(x_k) - f^*)^2}{\|\nabla f(x_k)\|^2} \geq \frac{(f(x_k) - f^*)^2}{2\beta(f(x_k) - f^*)}$$

$$\frac{(f(x_k) - f^*)^2}{\|\nabla f(x_k)\|^2} \geq \frac{f(x_k) - f^*}{2\beta}$$

$$\|x_{k+1} - x^*\|^2 \leq \|x_k - x^*\|^2 - \frac{1}{2\beta} (f(x_k) - f^*)$$

$$\sum_{i=0}^{k-1} \|x_{i+1} - x^*\|^2 \leq \sum_{i=0}^{k-1} \|x_i - x^*\|^2 - \frac{1}{2\beta} \sum_{i=0}^{k-1} (f(x_i) - f^*)$$

$$\sum_{i=0}^{k-1} \|x_{i+1} - x^*\|^2 - \sum_{i=0}^{k-1} \|x_i - x^*\|^2 \leq -\frac{1}{2\beta} \sum_{i=0}^{k-1} (f(x_i) - f^*)$$

$$\|x_k - x^*\|^2 - \|x_0 - x^*\|^2 \leq -\frac{1}{2\beta} \sum_{i=0}^{k-1} (f(x_i) - f^*)$$

$$\frac{1}{2\beta} \sum_{i=0}^{k-1} (f(x_i) - f^*) \leq \|x_0 - x^*\|^2 - \|x_k - x^*\|^2$$

$$\|x_0 - x^*\|^2 - \|x_k - x^*\|^2 \leq \|x_0 - x^*\|^2 \quad (\text{As we subtract a nonnegative value } \|x_k - x^*\|^2)$$

$$\frac{1}{2\beta} \sum_{i=0}^{k-1} (f(x_i) - f^*) \leq \|x_0 - x^*\|^2$$

$$\sum_{i=0}^{k-1} (f(x_i) - f^*) \leq 2\beta \|x_0 - x^*\|^2$$

$$\frac{1}{k} \sum_{i=0}^{k-1} (f(x_i) - f^*) \leq \frac{2\beta \|x_0 - x^*\|^2}{k}$$

$$f\left(\frac{1}{k} \sum_{i=0}^{k-1} x_i\right) \leq \frac{1}{k} \sum_{i=0}^{k-1} f(x_i) \quad (\text{By convexity of } f)$$

$$f\left(\frac{1}{k} \sum_{i=0}^{k-1} x_i\right) - f^* \leq \frac{1}{k} \sum_{i=0}^{k-1} (f(x_i) - f^*)$$

$$f\left(\frac{1}{k} \sum_{i=0}^{k-1} x_i\right) - f^* \leq \frac{1}{k} \sum_{i=0}^{k-1} (f(x_i) - f^*)$$

Putting it all together:

$$\begin{aligned} f\left(\frac{1}{k} \sum_{i=0}^{k-1} x_i\right) - f^* &\leq \frac{1}{k} \sum_{i=0}^{k-1} (f(x_i) - f^*) \\ &\leq \frac{2\beta \|x_0 - x^*\|^2}{k} \end{aligned}$$

That gives:

$$f\left(\frac{1}{k} \sum_{i=0}^{k-1} x_i\right) - f^* \leq \frac{2\beta \|x_0 - x^*\|^2}{k}$$

For α strongly convex f :

$$f(x_k) - f^* \geq \frac{\alpha}{2} \|x_k - x^*\|^2$$

$$\|\nabla f(x_k)\|^2 \leq 2\beta (f(x_k) - f^*)$$

$$f(x_{k+1}) - f^* \leq f(x_k) - f^* - \frac{(f(x_k) - f^*)^2}{4\beta (f(x_k) - f^*)}$$

$$f(x_{k+1}) - f^* \leq f(x_k) - f^* - \frac{f(x_k) - f^*}{4\beta}$$

$$f(x_{k+1}) - f^* \leq \left(1 - \frac{1}{4\beta}\right) (f(x_k) - f^*)$$

$$\frac{\alpha}{2} \|x_{k+1} - x^*\|^2 \leq \left(1 - \frac{\alpha}{4\beta}\right) \frac{\alpha}{2} \|x_k - x^*\|^2$$

$$\|x_{k+1} - x^*\|^2 \leq \left(1 - \frac{\alpha}{4\beta}\right) \|x_k - x^*\|^2$$

Acknowledgment: For this question, I got help from Batool.

Extra Credit: After completing the exercise, I asked ChatGPT to do the question and this is what it stated:

Assume f is also α -strongly convex.

From strong convexity,

$$f(x_k) - f^* \geq \alpha \|x_k - x^*\|^2$$

Here, ChatGPT makes the mistake of saying $f(x_k) - f^* \geq \alpha \|x_k - x^*\|^2$, which is untrue. Rather, $f(x_k) - f^* \geq \frac{\alpha}{2} \|x_k - x^*\|^2$.